Лабораторная работа
ПРОГРАММИРОВАНИЕ НА PYTHON В JUPYTER NOTEBOOK

Цель работы

Изучить основные приемы написания и документирования кода программы на Python с использованием *Jupyter Notebook*. Научиться создавать блокноты, открывать их на чтение и редактирование, обрабатывать содержащуюся в них информацию, вставлять ссылки, формулы, изображения и видео, выполнять в интерактивном режиме скрипты, написанные на языке Python.

Общие сведения о Jupyter Notebook

Для эффективной разработки программ на языке Python помимо *PyCharm*, считающейся самой интеллектуальной Python IDE с полным набором средств, часто используют *Jupyter Notebook*, который является развитием проекта IPython[1]. Это очень распространенное кроссплатформенное приложение, которое применяется повсеместно разработчиками на Python, а также учеными для презентации своей работы, интерактивного показа текста или фрагментов исходного кода программы на Python и результатов проведенной работы. Система *Jupyter* в отличие от IPython поддерживает более 100 языков программирования (называемых «ядрами»), включая Python, Java, R, Julia, Scheme, Scala, MatLAB и др., однако наиболее распространен вариант использования *Jupyter Notebook* для Python. Выполнить тестовый запуск полноценной версии *Jupyter* можно на сайте: https://try.jupyter.org.

Jupyter Notebook – невероятно мощный инструмент для интерактивной разработки, представления данных и их анализа. В нем можно разрабатывать, документировать и выполнять приложения на языке Python. Он состоит из двух компонентов:

---

[1] IPython (англ. *Interactive Python*) – интерактивная оболочка для языка программирования Python, которая предоставляет расширенную интроспекцию (возможность запросить тип и структуру объекта во время выполнения программы), дополнительный командный синтаксис, подсветку кода и автоматическое дополнение

– *веб-приложение*, запускаемое в браузере и позволяющее: 1) редактировать и запускать код на языке Python в браузере с подсветкой синтаксиса, автоотступами и автодополнением кода; 2) отображать результаты вычислений с медиа представлением (схемы, графики); 3) работать с языком разметки Markdown и LaTeX.
– *ноутбуки* (блокноты) – файлы с расширением *ipynb*, которые объединяют повествовательный текст, визуализацию, математические уравнения, мультимедиа, исходный код, входные и выходные данные в единый документ; ноутбуки можно экспортировать в форматы *pdf* и *html*.

Таким образом, *Jupyter Notebook* представляет собой не статическую страницу, а интерактивную среду, которая позволяет писать и выполнять код на Python посредством веб-интерфейса.

Установка Jupyter Notebook

Есть несколько вариантов установки *Jupyter Notebook* на компьютер. Наиболее распространенным и рекомендуемым вариантом для новичков является установка дистрибутива *Anaconda*, в составе которого присутствует *Jupyter Notebook* и который автоматически ставится при установке дистрибутива.

Anaconda – это широко распространенный дистрибутив для использования Python в сфере анализа данных. Его основное преимущество в том, что при его установке Python инсталлируется вместе с большинством пакетов, необходимых для обработки данных и машинного обучения (например, *Pandas* и *Numpy*), и библиотеками для создания визуализаций (рис. 1).



Рис. 1. Репозиторий дистрибутива *Anaconda*

Anaconda избавляет от необходимости самостоятельно устанавливать и настраивать среду разработки для Python, а

также от необходимости добавлять вручную в Python дополнительные библиотеки.

Рассмотрим процесс установки *Anaconda* для ОС *Windows*.

Шаг 1. Скачаем дистрибутив с официального сайта *Anaconda*: https://www.anaconda.com/distribution/#windows, воспользовавшись кнопкой *Download* (Скачать).

На момент подготовки материалов предлагалась 64-битная версия с установленным Python 3.9 (рис. 2).

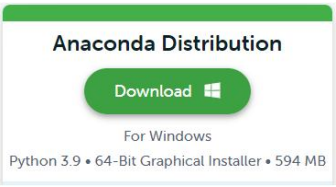


Рис. 2. Кнопка для загрузки *Анаконда* с официального сайта

Замечание. Чаще всего на современных компьютерах установлена 64-разрядная ОС, но для уверенности, перед установкой можно проверить ее разрядность в свойствах системы: → *Панель управления* → *Система*.

Шаг 2. После скачивания запускаем установочный файл и следуем инструкциям в установщике:

- принимаем лицензионное соглашение, нажав на кнопку *I Agree* (рис. 3а);
- выбираем тип установки: для текущего пользователя (рекомендуется оставить этот вариант) или для всех пользователей (рис. 3б);
- выбираем путь установки (рис. 3в); здесь следует учесть, что путь к *Anaconda* должен содержать только латинские символы, поэтому если пользователь *Windows* носит русскоязычное название, то необходимо установить *Anaconda* в другую папку;
- в заключительной инструкции рекомендуется: 1) НЕ добавлять *Anaconda* в переменную окружения PATH; 2) зарегистрировать *Anaconda* в качестве Python, который будет использоваться в системе по умолчанию (рис. 3г); оставляем настройки без изменения и нажимаем кнопку *Install* для запуска процесса установки.

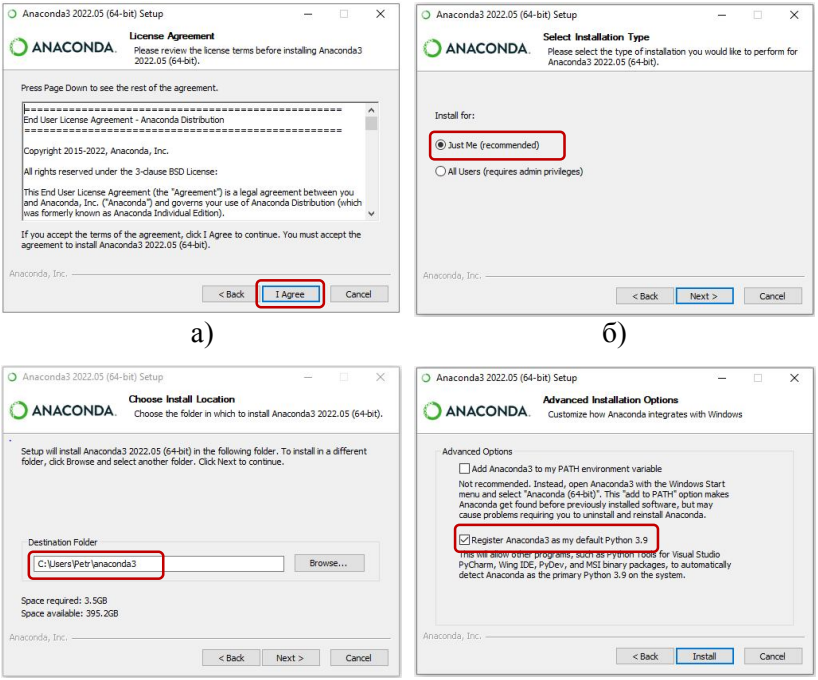


Рис. 3. Диалоги, отображаемые при установке *Anaconda*

После окончания установки в стартовом меню «Пуск» появится программная папка *Anaconda3* (*64-bit*) (рис. 4).

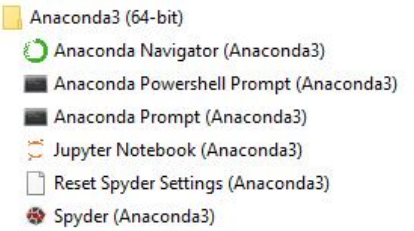


Рис. 4. Содержимое папки *Anaconda* стартового меню

Замечание. Если установщик выдает ошибку о том, что пользователь не имеет доступа к записи в указанную при инсталляции папку, то нужно перезапустить установщик с правами администратора.

Терминология Jupyter Notebook

Непосредственно перед знакомством с *Jupyter Notebook* необходимо разобраться с терминами, которые в нем используются и являются ключом, как к пониманию *Jupyter Notebook*, так и к тому, что делает его не просто текстовым редактором.

Markdown – это язык разметки текстов, являющийся надмножеством HTML, предназначенный для создания красиво оформленных текстов в обычных текстовых файлах (формата *txt*), а также для конвертации текстов в формат *html* или *pdf*.

Смысл использования Markdown в том, что при подготовке текста на его разметку тратят минимум усилий, а уже какой-то другой плагин или программа, поддерживающая язык разметки Markdown, превращает эту разметку в итоговый документ (например, в *html*). Markdown – это как бы язык для других программ (метаязык), передающий им правила формирования документов на основе вашего текста.

Пример. В Markdown для оформления заголовков используют решетку (#): одна решетка определяет заголовок первого уровня, две решетки – заголовок второго уровня, и т.д. до пятого уровня. При подготовке текста фрагменты, соответствующие заголовкам, будут размечены следующим образом:

```
# Глава 1. Программы с линейной структурой
---
## 1.1. Линейный алгоритм
Линейный алгоритм включает последовательность ко-
манд (указаний, действий, операторов), выполняемых
друг за другом в естественном порядке.
```

После преобразования разметки Markdown итоговый документ будет имет вид, показанный на рис. 5.

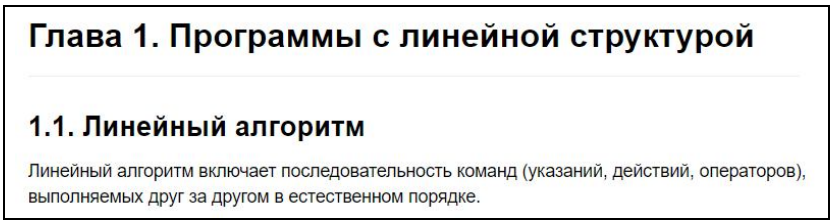


Рис. 5. Вид итогового документа

Как видно из примера, три подряд идущие знака «минус» преобразовались в горизонтальную линию.

Если при наборе теста мы просто разметим его с помощью Markdown, то он так и останется текстом, содержащим различные спецсимволы. Чтобы в результате текст, размеченный специальными символами, оказался отформатированным, используют специальные программы (плагины), поддерживающие язык разметки Markdown. *Jupyter Notebook* поддерживает язык разметки Markdown.

Kernel (ядро) – это «вычислительный движок», который запускает и проводит внутренний анализ кода, содержащегося в ноутбуке, и контролирует все вычисления и вывод их результатов на экран. Каждый язык программирования использует отдельное ядро.

Cell (ячейка) – это контейнер для *текста*, который будет отображаться в ноутбуке, или *кода*, который будет выполняться ядром, поэтому выделяют два основных типа ячеек: `Code` и `Markdown`. Ячейки образуют структуру ноутбука.

Ячейка типа `Code` содержит код, который должен быть выполнен в ядре, и отображает результат его выполнения ниже.

Ячейка типа `Markdown` содержит текст, отформатированный с использованием Markdown, и отображает его вывод на месте при запуске.

Запуск Jupyter

Запустить *Jupyter* в *Windows* после установки *Anaconda* можно двумя способами. Вы можете выбрать любой способ из представленных.

Первый способ (самый быстрый) – запустив программу из меню *Пуск* → *Все программы* → *Anaconda3* (*64-bit*) → *Jupyter Notebook* (*Anaconda3*).

Второй способ – через *Anaconda Navigator*, запустив программу из меню *Пуск* → *Все программы* → *Anaconda3* (*64-bit*) → *Anaconda Navigator* (*Anaconda3*). После запуска появится окно программы *Anaconda Navigator* с выбором приложения. Запустить *Jupyter* можно, нажав на кнопку *Launch* на плитке приложения *Jupyter Notebook* (рис. 6).

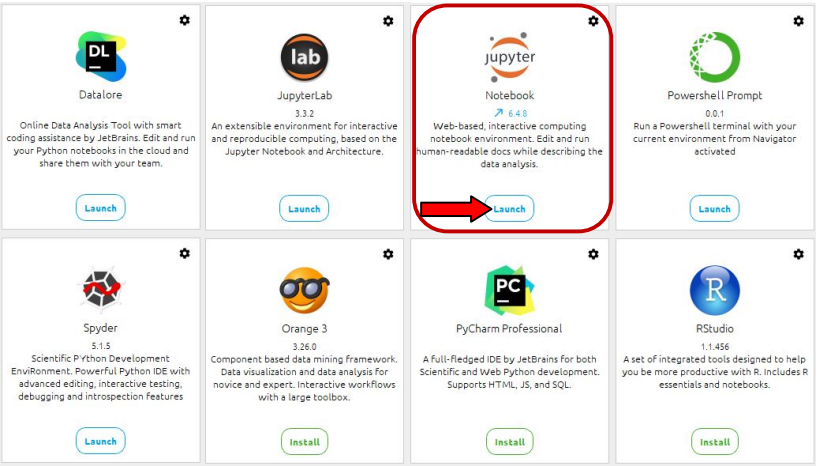


Рис. 6. Плитка приложений программы *Anaconda*

Третий способ – через *Anaconda Prompt*: запустив программу из меню *Пуск* → *Все программы* → *Anaconda3* (*64-bit*) → *Anaconda Prompt* (*Anaconda3*). После выбора приложения отобразится окно терминала, в котором для запуска *Jupyter* нужно ввести команду `jupyter notebook` (рис. 7).

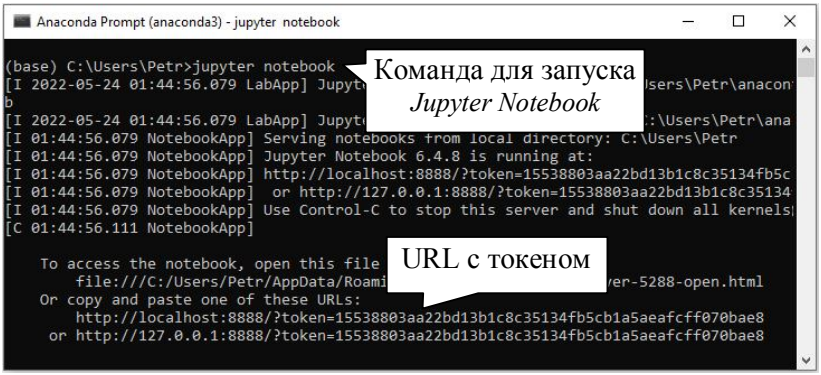


Рис. 7. Окно терминала программы *Anaconda Prompt*

В процессе запуска любым из способов создается локальный веб-сервер *Jupyter*, прослушивающий сетевой порт с номером 8888. Автоматически в веб-браузере, используемым по умолчанию, откроется новая вкладка *Home* с оболочкой *Jupyter*

(рис. 8) со следующим URL: http://localhost:8888/tree. Это *панель управления* ноутбуками *Jupyter*.

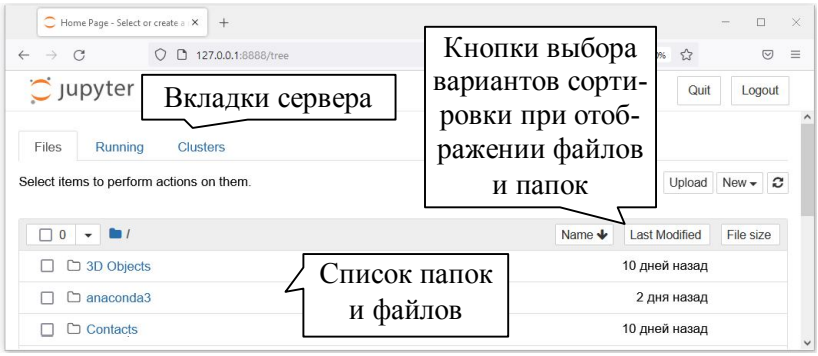


Рис. 8. Вид окна браузера после запуска *Jupyter Notebook*

Localhost (иначе 127.0.0.1) – это адрес локального компьютера; *localhost* не является веб-сайтом, но указывает, что контент обслуживается с вашего собственного компьютера.

Панель управления предоставляет доступ только к файлам и подпапкам, содержащимся в рабочем каталоге *Jupyter*, из которого он запускается. Этот каталог определяется при установке, но может быть изменен.

Замечание. Если вы используете по умолчанию браузер, который запрещает появление всплывающих окон, то *Jupyter* может не открыться в браузере. В этом случае в окне консоли, появившемся при запуске (рис. 7), необходимо вручную скопировать строку с адресом (http://localhost:8888/tree) и токеном, после чего вставить в адресную строку браузера.

Обратите внимание! Окно консоли, появившееся после запуска локального веб-сервера (рис. 7), закрывать нельзя, иначе *Jupyter* перестанет работать. Правильно завершить работу *Jupyter* можно, дважды нажав в этом окне комбинацию <Ctrl>+<C>, а не закрыть его, нажав на крестик ×.

Создание и переименование папки

Для хранения своих ноутбуков создадим отдельную папку, нажав на кнопку *New* в правой части панели управления вкладки *Files* и выбрав в выпадающем списке команду *Folder* (рис. 9).



Рис. 9. Создание новой папки, файла или терминала

Новая папка получит название «*Untitled Folder*». Поставив галочку напротив этой папки, можно воспользоваться кнопкой *Rename* для ее переименования (рис. 10). Новое имя папки вводится в появившемся окне *Переименовать директорию*.

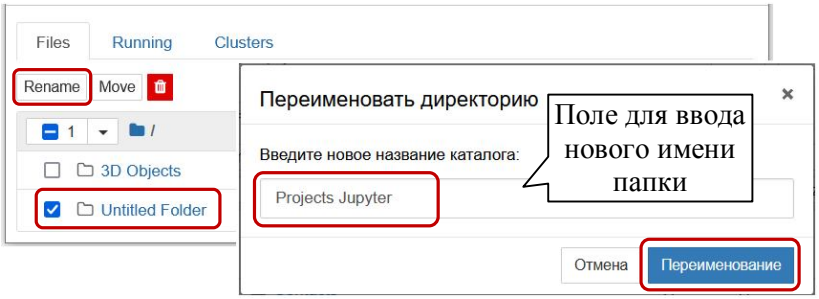


Рис. 10. Переименование папки

Создание и сохранение блокнота

Для создания ноутбука – документа *Jupyter Notebook* с текстом первой программы перейдем в созданную папку (выполнив двойной клик по ней в браузере), нажмем на кнопку *New* в правой части панели управления и выберем в выпадающем списке команду *Python3* (рис. 9). В результате будет создан ноутбук, а окно браузера примет вид, показанный на рис. 11.

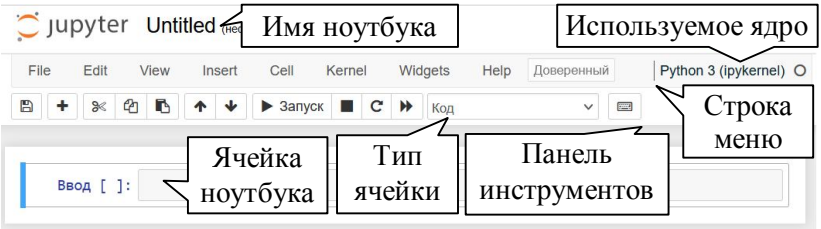


Рис. 11. Интерфейс ноутбука

Вверху страницы отображается название ноутбука «*Untitled*» (заданное по умолчанию), оно отображается и в заголовке страницы. Под названием располагаются строка команд меню и панель инструментов *Jupyter Notebook* (рис. 11).

Переименуем ноутбук. Для этого достаточно навести на его название мышь и кликнуть по тексту. В появившемся окне *Переименовать блокнот* можно ввести новое имя ноутбука (рис. 12) и нажать на кнопку *Переименование*.

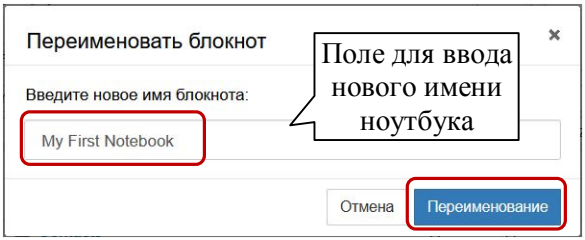


Рис. 12. Диалоговое окно переименования ноутбука

Отличительной особенностью написания кода в ноутбуке является то, что код разбивается на ячейки, каждая из которых может быть выполнена отдельно.

В созданном ноутбуке по умолчанию содержится единственная ячейка типа `Code` (Код) и эта ячейка использует ядро, которое было выбрано при его запуске. Перед ячейкой написано `Ввод []` (`In []`). Это ключевое слово значит, что здесь ожидается ввод кода программы. Синий контур, обрамляющий ячейку, означает командный режим. Если установить курсор внутрь ячейки, то цвет подсветки контура изменится на зеленый, это будет означать, что ячейка находится в режиме редактирования.

Ноутбук может находиться в одном из двух режимов работы с ячейками:

- *режим правки* (*Edit mode*) – когда происходит редактирование содержимого, копирование, вставка; есть возможность возвращаться к ячейкам и оперативно вносить правки;
- *командный режим* (*Command mode*) – когда происходит выполнение команд, результат выводится отдельно по каждой команде.

Текущий режим отображается в правой части строки меню (рис. 13). В режиме правки появляется изображение карандаша; отсутствие которого показывает, что ноутбук находится в командном режиме.

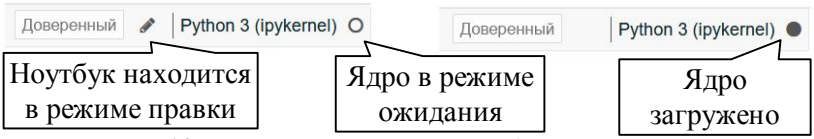


Рис. 13. Индикаторы режима работы с ячейками и загруженности ядра

Рядом с индикатором режима работы с ячейками находится индикатор загруженности ядра Python (рис. 13). Если ядро находится в режиме ожидания, то индикатор представляет собой окружность. Если ядро выполняет какую-то задачу, то изображение измениться на закрашенный круг. При выполнении достаточно простых операций обычно незаметно, как кружок меняет цвет. Но если Python выполняет какую-то затратную операцию (например, сложный алгоритм с большим числом итераций), индикатор достаточно долго может оставаться черным. Такую же картину можно увидеть, если Python вдруг завис.

В *Jupyter Notebook* можно делать все то же, что и в интерактивном интерпретаторе Python и в PyCharm. Ноутбук можно сохранять, можно вернуться к нему в дальнейшем, перезапустить ячейки, – это очень удобно.

В ячейке `Ввод []` блокнота *Jupyter Notebook* наберем код решения задачи нахождения первых 10 чисел последовательности Фибоначчи на языке Python (рис. 14).

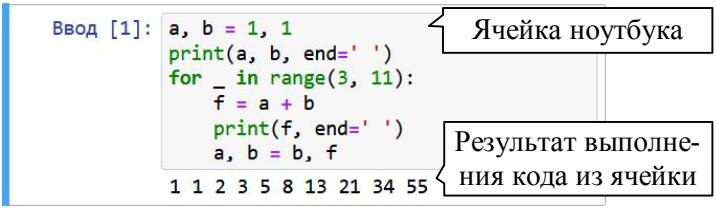


Рис. 14. Код программы с результатами работы

Для запуска кода из активной ячейки воспользуемся кнопкой *Запуск* (*Run*) ▶ Запуск панели инструментов *Jupyter Notebook*.

Для удобства можно запустить ячейку на выполнение, используя следующие комбинации клавиш:

– <Ctrl>+<Enter> – запуск кода активной ячейки на выполнение (ячейка остается активной);
– <Alt>+<Enter> – запуск кода активной ячейки на выполнение и добавление новой ячейки под пекущей (активной становится добавленная ячейка);
– <Shift>+<Enter> – запуск кода активной ячейки на выполнение и переход к следующей (если под активной ячейкой нет ячеек, то добавляется новая).

Если в коде запускаемой на выполнение ячейки предусмотрен вывод результатов, то *Jupyter Notebook* отобразит вывод под ячейкой. Это очень удобно и позволяет заниматься программированием в интерактивном режиме, имея возможность отслеживать вывод каждого шага программы и тестировать каждый шаг, оперативно внося изменения при обнаружении неполадок.

Как показывает рис. 14, после запуска ячейки вместо `Ввод []` теперь написано `Ввод [1]`. Появление номера (числа) в скобках строки `Ввод []` означает, что код ячейки выполнен, а число `N` в квадратных скобках показывает номер выполненной ячейки. Последовательность номеров не строгая.

Наличие символа «звездочка» в скобках этой строки (`Ввод [*]`) означает, что ячейка выполняется (рис. 15).

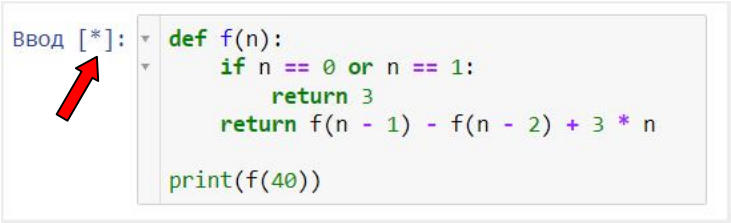


Рис. 15. Ячейка в режиме выполнения

Код в ячейке можно модифицировать и запускать повторно (изменится индексация ячеек), т.е. можно изменять очередность выполнения ячеек. Но при этом необходимо следить за тем, **чтобы интерпретатор не обращался к переменным, которые на данный момент он не знает**.

На рис. 16, слева показан результат запуска ячейки с неопределенной переменной, при котором возникло исключение `NameError`. На рис. 16, справа показан запуск ячеек с непоследовательным порядком выполнения: первой была выполнена центральная ячейка, потом верхняя, далее нижняя. Если повторить после этих действий запуск, например верхней ячейки, то в результат будет равен 17.

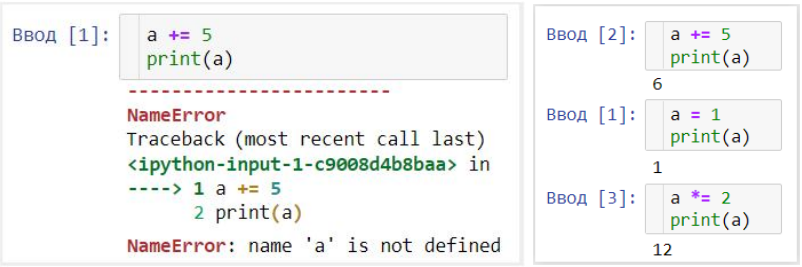


Рис. 16. Запуск ячейки с неопределенной переменной (слева) и с измененным порядком следования ячеек (справа)

Данный пример показывает, что *Jupyter Notebook* **поддерживает нелинейную последовательность исполнения**, т.е. он позволяет запускать ячейки с кодом в произвольном порядке. Конечно, это повышает требования к разработчику, который должен помнить, какая конкретно последовательность ячеек решает задачу. Но часто нелинейная последовательность исполнения оказывается удобной и востребованной, например, когда полный перезапуск решения требует нескольких часов, а надо перезапустить только определенный участок кода.

Чтобы добавить новую ячейку, можно воспользоваться кнопкой + панели инструментов. Новая ячейки будет добавлена под активной.

Jupyter Notebook поддерживает групповые операции с ячейками. Для выделения группы подряд идущих ячеек используется <Shift>+клик на полях слева от ячеек.

Порядок следования ячеек можно изменить с помощью кнопок ↑ / ↓, которые перемещают выбранные ячейки соответственно вверх / вниз.

Каждый ноутбук при открытии использует новую вкладку. При этом создается файл ноутбука с расширением *ipynb*, а также

файл контрольной точки. Файл контрольной точки будет расположен в скрытом подкаталоге с именем *.ipynb_checkpoints* в месте сохранения исходного ноутбука, он также является файлом *.ipynb*. Открытые ноутбуки автоматически сохраняются каждые 120 секунд (поэтому данные теряются довольно редко), обновляя только исходный файл *.ipynb*, а не файл контрольной точки.

При выборе команды *File → Save and Checkpoint* (*Файл → Сохранение и контрольная точка*) или комбинации <Ctrl>+S файлы ноутбука и контрольной точки обновляются. Следовательно, контрольная точка позволяет восстановить ноутбук в случае непредвиденной проблемы, т.е. **контрольная точка** – это резервное хранилище на случай форс-мажоров. Для сохранения ноутбука и файла контрольной точки используется также кнопка  панели инструментов.

Вернуться к контрольной точке можно через меню *File → Revert to Checkpoint* (*Файл → Вернуться к контрольной точке*). При выборе этой команды в диалоге возврата к контрольной точке необходимо подтвердить свои действия (рис. 17).



Рис. 17. Диалог возврата к контрольной точке

Если после последнего сохранения в ноутбук не были внесены изменения, а пользователь закрывает страницу браузера, то будет выведено всплывающее сообщение с подтверждением совершаемых действий (рис. 18).

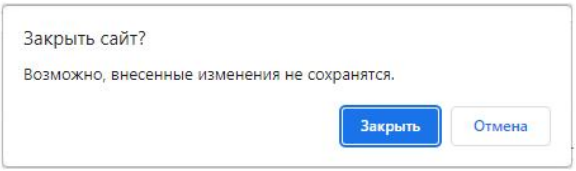


Рис. 18. Сообщение, возникающее при закрытии браузера

Если нужно загрузить копию файла с другим расширением, например, как *.py*, необходимо выбрать меню *File* → *Download* as → *Python* (*.py*) (*Файл* → *Загрузить как* → *Python* (*.py*)).

Интерфейс Jupyter Notebook

Горячие клавиши Jupyter Notebook

При работе в *Jupyter Notebook* всегда есть одна *активная* ячейка, выделенная рамкой, цвет которой обозначает ее текущий режим, где зеленый означает режим редактирования, а синий – командный режим. Переключение между режимом редактирования и командным режимом осуществляется с помощью клавиш <Esc> и <Enter> соответственно. Важно понимать в каком режиме находится активная ячейка, поскольку в командном режиме для ускорения рабочего процесса можно использовать сочетания клавиш, которые являются очень популярным средством *Jupyter Notebook*. В табл. 1 представлены часто используемые при работе в *Jupyter Notebook* клавиши (и их комбинации), их назначение.

Таблица 1. Часто используемые при работе клавиши

Сочетание клавиш	Режим работы	Назначение
`Ctrl+S`	Любой	Сохранить ноутбук
`Tab`	Редактирование	Завершение кода или отступ
`Shift+Tab`	Редактирование	Всплывающая подсказка
`Esc`	Редактирование	Вход в командный режим, после этого при помощи стрелок можно выбирать другую ячейку
`Ctrl+Shift+-`	Редактирование	Разделение активной ячейки по курсору
`Shift+Enter`	Командный	Выполнение активной ячейки + переключение фокуса на следующую существующую
`Alt+Enter`	Командный	Выполнение активной ячейки + создание новой ячейки снизу + переключение фоку-

		са на нее
`Ctrl+Enter`	Командный	Выполнение активной ячейки, фокус остается на ней же
`A`	Командный	Создать пустую ячейку над активной ячейкой (от *above*)
`B`	Командный	Создать пустую ячейку под активной ячейкой (от *below*)
`D+D`	Командный	Удалить активную ячейку (нужно нажать *d* два раза)
`Z`	Командный	Отменить удаление ячейки
`O`	Командный	Скрыть/ отобразить вывод активной ячейки
`M`	Командный	Изменить тип активной ячейки на `Markdown`
`Y`	Командный	Изменить тип активной ячейки на `Code`
`L`	Командный	Включить/ выключить нумерацию строк в ячейке

Перечень встроенных в *Jupyter Notebook* сочетаний клавиш (рис. 18) доступен по нажатию комбинации <Esc>+<H> или из меню *Help → Keyboard Shortcuts* (*Справка → Горячие клавиши*).

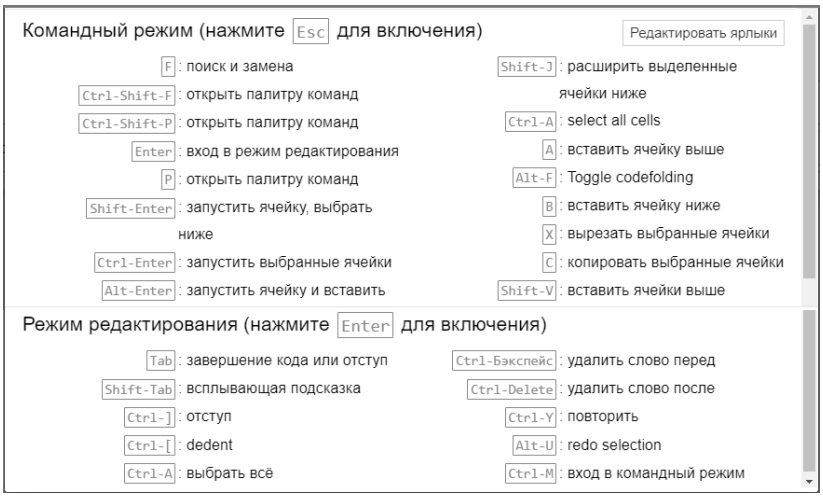


Рис. 19. Окно со встроенными сочетаниями клавиш

Получение справочной информации

В *Jupyter Notebook* встроена мощная система помощи. На рис. 20 показана всплывающая подсказка, которая отображается при нажатии комбинации <Shift> + <Tab> во время ввода оператора `print`. Кнопка + раскрытия текста подсказки позволяет не только отобразить информацию по интересующему объекту языка, но и посмотреть пример его использования.

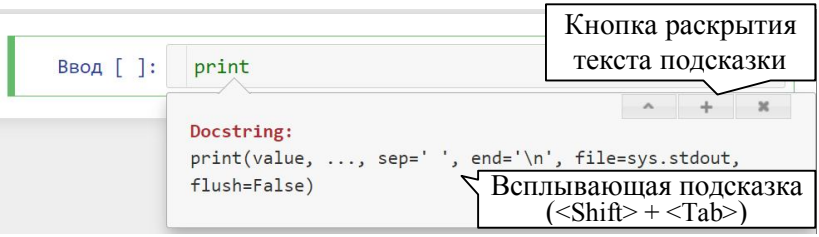


Рис. 20. Всплывающая подсказка

Отобразить общую информацию об объекте (интроспекция объекта) можно также с помощью ? или ??. Для функций ? показывает строку документации, ?? – по возможности показывает исходный код функции (рис. 21).

```
Ввод [1]: lst = [1, 5, -8, 6, 17, -9]
          lst?
```

```
Type:        list
String form: [1, 5, -8, 6, 17, -9]
Length:      6
Docstring:
Built-in mutable sequence.

If no argument is given, the constructor creates a new empty list.
The argument must be an iterable if specified.
```

Рис. 21. Вывод информации об объекте

В *Jupyter Notebook* имеется ряд полезных команд. Отобразить командную панель можно с помощью кнопки панели инструментов со значком клавиатуры ⌨ (<Ctrl> + <Shift> + <P>).

Панель инструментов

Кнопки, формирующие панель инструментов *Jupyter Notebook*, помогают выполнять часто используемые команды редактора. В табл. 2. приведено их назначение.

Таблица 2. Назначение кнопок панели инструментов

Кнопка	Назначение
	Сохранение файла ноутбука и файла контрольной точки
	Добавление новой ячейки. Новая ячейки будет добавлена под активной
	Вырезать выделенные ячейки (содержимое ячеек помещается в буфер обмена, сама ячейка удаляется)
	Скопировать выделенные ячейки (содержимое ячеек помещается в буфер обмена, сама ячейка остается)
	Вставить ячейку из буфера ниже выделенной ячейки
	Перемещение выбранных ячеек соответственно вверх или вниз
	Запуск кода из активной ячейки на выполнение с последующим переходом к следующей ячейке
	Прерывание работы ядра. Ели в этот момент выполнялась ячейка с кодом, то выбрасывается исключение `KeyboardInterrupt`
	Перезапуск ядра с очисткой значений всех переменных, которые были определены в текущем сеансе работы
	Перезапуск ядра с последующим переходом к последней ячейке
	Выбор типа ячейки: `Код`, `Markdown`, `Необработанный NBConvert` и `Заголовок`
	Отображение командной панели с перечнем доступных команд и горячими комбинациями их вызова

Замечание. Технически существует четыре типа ячеек: `Код`, `Markdown`, `Необработанный NBConvert` и `Заголовок`. Но тип ячейки `Заголовок` больше не поддерживается и в случае его выбора отобразится диалоговое окно с соответствующим сообщением и предложением использовать вместо этого `Mark-`

down для своих заголовков. Основные используемые типы ячеек – это типы ячеек Code и Markdown. Тип ячеек Необработанный NBConvert предназначен для написания неразмеченного текста и для особых случаев использования nbconvert инструмента командной строки, который предназначен для преобразования или экспорта вашего ноутбука в другой формат (*html*, *pdf*, *LaTeX* и др.).

Строка меню Jupyter Notebook

Строка меню *Jupyter Notebook* (рис. 22) содержит команды, обеспечивающие взаимодействие с ноутбуком, доступ к документации популярных библиотек Python, экспорта проекта для последующей демонстрации и пр.



Рис. 22. Строка меню

Опишем основное назначение пунктов строки меню.

В меню *File* (*Файл*) расположены команды для создания нового блокнота, переименования и открытия уже существующего, его экспорта в один из доступных форматов файлов. Здесь же создаются контрольные точки, к которым можно откатиться в случае возникновения нештатных ситуаций.

Команды меню *Edit* (*Правка*) предназначены для выполнения операций с ячейками. С их помощью можно вырезать, копировать, вставлять, удалять, разделять или объединять ячейки, вставлять в ячейку изображение, а также изменять порядок следования ячеек в ноутбуке.

Замечание. Некоторые элементы в этом меню могут оказаться недоступными (пункт отображается серым цветом). Причина в том, что недоступные команды не применяются к типу выбранной ячейки. Например, в ячейку типа Code нельзя вставить изображение, а в ячейку типа Markdown можно. Если вы видите пункт меню серого цвета, попробуйте изменить тип ячейки и посмотрите, станет ли этот элемент доступным для использования.

В меню *View* (*Вид*) расположены команды для переключения видимости строки заголовка и панели инструментов, а также нумерации строк в ячейках.

Меню *Insert* (*Вставка*) предназначено только для вставки ячеек выше или ниже выбранной ячейки.

Команды меню *Cell* (*Ячейка*) позволяет запускать одну ячейку, группу ячеек, сразу все ячейки, можно запускать ячейки в определенном порядке. Здесь можно менять тип ячеек. Другая удобная функция этого меню – очистка вывода текущей ячейки и вывода всех ячеек ноутбука.

Используя команды меню *Kernel* (*Ядро*) можно запускать, прерывать, перезапускать или выключать ядро, а также запускать новое ядро. Когда вы запускаете ячейку кода, этот код выполняется в ядре, и любой вывод возвращается обратно в ячейку для отображения. Состояние ядра сохраняется во времени и между ячейками – оно относится к документу в целом, а не к отдельным ячейкам.

Вероятно, вы не будете так часто работать с ядром, но бывают случаи, когда вы отлаживаете ноутбук, и вам могут пригодиться следующие команды меню *Kernel*:

- *Restart* – перезапускает ядро, очищая все переменные (и прочие объекты), которые были определены;
- *Restart & Clear Output* – то же, что и *Restart*, но дополнительно стирает вывод, отображаемый под ячейками кода;
- *Restart & Run All* – то же, что и *Restart*, но дополнительно запускает все ячейки от первой до последней на выполнение;
- *Interrupt* – прерывает выполнение ячейки, бывает полезно, если ваше ядро зависло в вычислении, и его требуется остановить.

Меню *Widgets* (*Виджеты*) содержит команды для сохранения и очистки состояния виджета (см. далее).

С помощью команд меню *Help* (*Справка*) можно познакомиться с пользовательским интерфейсом *Jupyter Notebook*, узнать о сочетаниях клавиш, получить справочные материалы по популярным библиотекам Python.

Приемы документирования в Jupyter Notebook

Jupyter Notebook поддерживает язык разметки Markdown, являющийся надмножеством HTML, в частности, поэтому при наборе текста несколько подряд идущих пробелов считаются за один пробел.

При написании размеченного текста, изменяем тип ячейки на `Markdown`, вводим текст с использованием разметки Markdown и компилируем, используя комбинацию <Shift> + <Enter>.

Приемы форматирования текста

При наборе текста в ячейке типа Markdown для его стилизации можно использовать следующие приемы:

- заголовки создают с помощью символа «#» и следующего за ним пробела; один такой символ сделает крупный заголовок верхнего уровня, два – заголовок поменьше и т.д. до шестого уровня;
- символ звездочка «*» с двух сторон текста нужен для оформления этого текста курсивом;
- два символа звездочка «**» с двух сторон текста позволяют придать тексту полужирное начертание;
- для создания маркированного списка, каждый его элемент начинают с символов «*», «–» или «+» и пробела;
- нумерованные списки начинают с цифры и точки, причём цифры могут быть любыми и идти не по порядку.

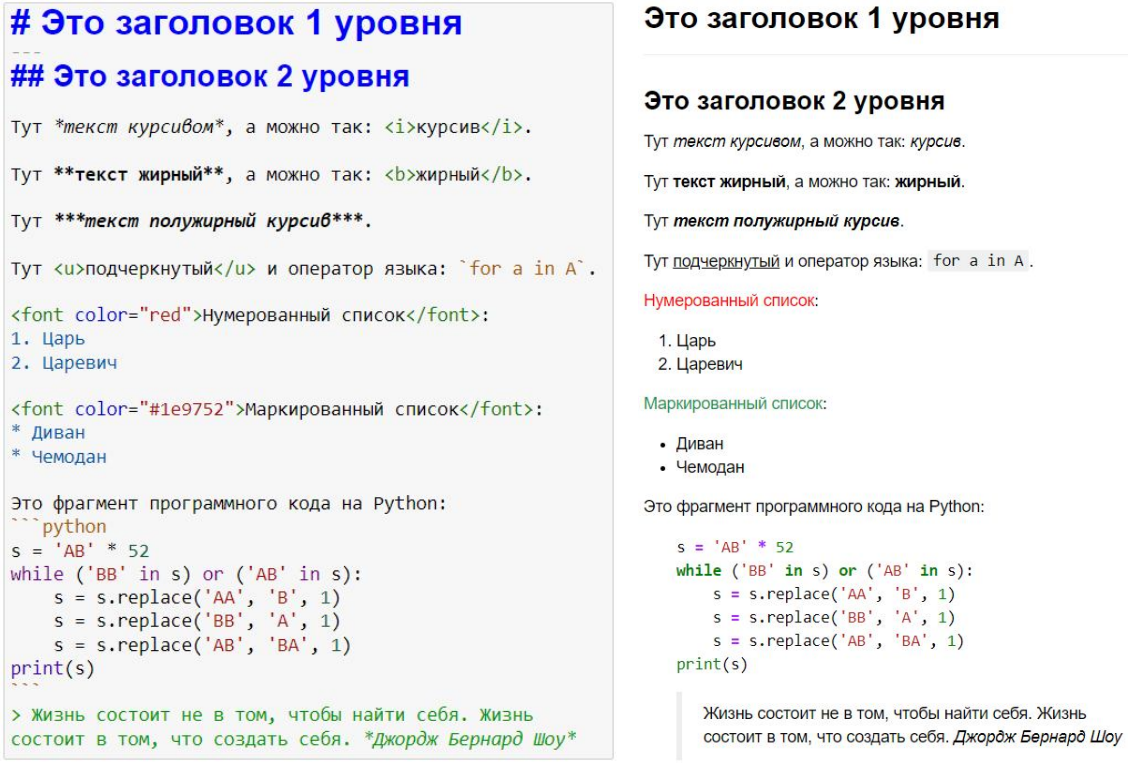
В табл. 3 представлена разметка Markdown, используемая при форматировании текста.

Таблица 3. Коды форматирования текста в разметке Markdown

Отображаемый текст	Код разметки	Примечание
Заголовок 1	`# Заголовок 1`	Заголовки должны начинаться с новой строки
Заголовок 2	`## Заголовок 2`	Заголовки занимают очень много места в окне. Самый маленький заголовок – 6 символов «#»

Курсив	*Курсив* _Курсив_	Символ «_» можно заменить на «*»
Жирный	**Жирный**	
Жирный курсив	***Жирный курсив***	
Не курсив #Не заголовок# **Не жирный**	_Не курсив_ \#Не заголовок\# **Не жирный**	Если надо написать символ, используемый в Markdown, то используйте экранирование символов – поставьте перед ним обратный слэш «\»
> Цитата	> Цитата	Символ «>» используется для многоуровневого цитирования
_____	---	Введите на новой строке три (или более) нижних подчёркивания «___» или три звездочки «***», перейдите на новую строку
`code`	`code`	Символ «`» обратного апострофа используется для оформления программного кода
• первый элемент; • второй элемент.	- первый элемент; - второй элемент.	Начните строку с «+», «*» или «-» и пробела
1. Первый элемент. 2. Второй элемент.	1. Первый элемент. 2. Второй элемент.	Начните строку с «1.» и пробела

Как видно из табл. 3, ячейку типа Markdown можно использовать, если требуется вставить фрагмент программного кода, который не предусматривает дальнейший запуск пользователем. В таком случае код заключают в обратные апострофы. Если требуется вставить блок кода, можно использовать тройные обратные апострофы, а также указать язык программирования, как показано на рис. 23, слева.



```
# Это заголовок 1 уровня
---
## Это заголовок 2 уровня

Тут *текст курсивом*, а можно так: <i>курсив</i>.

Тут **текст жирный**, а можно так: <b>жирный</b>.

Тут ***текст полужирный курсив***.

Тут <u>подчеркнутый</u> и оператор языка: `for a in A`.

<font color="red">Нумерованный список</font>:
1. Царь
2. Царевич

<font color="#1e9752">Маркированный список</font>:
* Диван
* Чемодан

Это фрагмент программного кода на Python:
```python
s = 'AB' * 52
while ('BB' in s) or ('AB' in s):
    s = s.replace('AA', 'B', 1)
    s = s.replace('BB', 'A', 1)
    s = s.replace('AB', 'BA', 1)
print(s)
```

> Жизнь состоит не в том, чтобы найти себя. Жизнь
состоит в том, что создать себя. *Джордж Бернард Шоу*
```

Рис. 23. Приемы форматирования текста (слева) и его отображение (справа) в Markdown

На рис. 23, слева показаны приемы форматирования текста в разметке Markdown, справа – отображение текста этой ячейки в браузере после компиляции (после нажатия <Shift> + <Enter>).

Поскольку язык разметки Markdown является надмножеством HTML, то при подготовке текста можно использовать *html*-теги, как показано на рис. 23, слева.

Добавление таблиц

Разметка Markdown позволяет вставлять таблицы. Для разметки таблицы используются всего два символа: вертикальная черта «|» и дефис «-». Дефисы работают примерно так же, как в случае с горизонтальной линией: отделяют заголовки от других строк, при этом количество символов значения не имеет. Вертикальная черта служит границей между столбцами. На рис. 24 приведен пример разметки таблицы и ее отображение в браузере после компиляции Markdown-ячейки.

```
| № п/п | Фамилия И.О. | Оценка |
|-|-|-|
| 1 |Иванов И.И.      | 5 |
| 2 |Школин А.А.      | 5 |
| 3 |Назаров Р.Ю.     | 4 |
| 4 |Грирорьева Л.Д.  | 3 |
| 5 |Наумов Н.П.      | 4 |
```

№ п/п	Фамилия И.О.	Оценка
1	Иванов И.И.	5
2	Школин А.А.	5
3	Назаров Р.Ю.	4
4	Грирорьева Л.Д.	3
5	Наумов Н.П.	4

Рис. 24. Разметка для таблицы (слева) и ее отображение (справа)

Вставка ссылок на Интернет-ресурсы

Есть несколько способов добавления в ноутбук ссылок на внешние Интернет-ресурсы, в зависимости необходимости отображения адреса самого ресурса и заголовка, который будет отображаться рядом с курсором при наведении на ссылку.

Чтобы добавить гиперссылку, нужно добавить текст ссылки, заключив его в квадратные скобки и рядом в круглых скобках поместить URL ресурса и при необходимости текст заголовка (между квадратными и круглыми скобками не должно быть пробела):

```
[Текст ссылки](URL «Необязательный заголовок»)
```

На рис. 25 приведен пример добавления ссылок в ноутбук.



Рис. 25. Добавление гиперссылок в ноутбук (слева) и их отображение (справа)

Вставка медиа-контента

Самый простой способ вставки в ячейку типа Markdown картинки с локального компьютера – воспользоваться меню *Edit → Insert Image* (*Правка → Вставить изображение*), указав в диалогах путь к файлу с изображением.

Синтаксис Markdown для вставки картинок очень похож на разметку, используемую для вставки ссылок. Разница в восклицательном знаке перед первыми квадратными скобками:

```
![Alt-текст изображения](Источник изображения
 «Необязательный заголовок изображения»)
```

В качестве `Источника изображения` необходимо указать локальный путь к файлу с изображением, который может быть абсолютным или относительным; здесь также можно указать URL изображения, например:

```
![Рыжий кот](Images/Image_1.jpg)
![Фото](http://www.rsreu.ru/modules/mod_imgslide/
              images/img1.jpg)
```

При вставке локального и сетевого изображения можно также использовать *html*-тег `<img src=”…”>`, где в атрибуте `src` указать путь к изображению, например:

```
<img src="./image.png", width=320, heigth=240>
<img src="http://www.rsreu.ru/modules/mod_imgslide/
      images/img1.jpg" height="1000" width="800">
```

Html-теги можно использовать и для вставки в ноутбук видеофайла, например можно использовать тег `<video controls src=”…”>`:

```
<video controls src="Video/video.mp4" width="600"
                    height="600">
```

В рассмотренном фрагменте кода видеофайл лежит в папке *Video*, в той же директории что и ноутбук (сам файл с расширением *.ipynb*).

Работа с формулами

Формулы, вставляемые в ячейки типа `Markdown`, должны быть подготовлены в соответствии с синтаксисом LaTeX.

LaTeX – система набора и верстки текстов с формулами, стандарт для обмена и публикации научных документов.

Формулу можно вставлять внутрь текста в строке или располагать на отдельной строке, по центру и с номером. Поэтому выделяют несколько режимов вставки формулы в ноутбук:

- ***встроенный режим*** – для вставки формулы использует синтаксис: `$ formula $`; в этом режиме при отображении формула будет смешана с текстом до и после нее;
- ***блочный режим*** – для вставки формулы использует синтаксис: `$$ formula $$`; в этом режиме при отображении формула выделяется в отдельную строку, выравнивается по центру и отделяется сверху и снизу пустыми строками;
- для добавления длинных и сложных математических выражений часто используется ***окружение*** `equation`: `\begin{equation}\label{ссылка} ... \end{equation}`, которое в LaTeX позволяет организовать автоматическую нумерацию формул и ссылок на них, но для использования этой возможности в *Jupyter Notebook* потребуется установить соответствующее расширение.

Формулами считаются как целые формулы, так и отдельные цифры или буквы, в том числе буквы греческого алфавита, а также верхние и нижние индексы и спецзнаки. Один пробел внутри исходного текста, задающего формулу, игнорируется.

При подготовке формул используются следующие **специальные символы**, которые не выводятся на экран:

- «{» – начало группы;
- «}» – конец группы;
- «_» – нижние индексы;
- «^» – верхние индексы;
- «\» – сигнальный символ команд.

Если эти символы должны сами присутствовать в формуле, то их следует экранировать и они будут отрисованы в формуле, например `$x \{j \}$` .

Рассмотрим пример набора формул, в котором показаны встроенный и блочный режимы вставки математических выражений. На рис. 26 приведен пример разметки, а на рис. 27 – отображение текста с формулой в браузере после компиляции ячейки.

```
Функция распознавания может быть записана в виде:

$$f(x,W)=h^{(2)}\left(\sum\limits_{i=1}^D
w_i^{(2)}h^{(1)}\left(\sum\limits_{j=1}^n
w_{ji}^{(1)}x_j+b_i^{(1)}\right)+b^{(2)}\right)$$

Здесь: $x$ – исходный объект (описанный $11$ признаками), $x_j$ –
соответствующий признак, $n$ – количество нейронов во входном
слое сети, совпадающее с количеством признаков, $D$ – количество
нейронов в скрытом слое сети, $w_i^{(2)}$, w_{ji}^{(1)},
b_i^{(1)}, b^{(2)}$ – параметры сети, соответствующие весам
нейронов, $h^{(1)}$, h^{(2)}$ – функции активации.
```

Рис. 26. Использование формата LaTeX при наборе формул

Функция распознавания может быть записана в виде:

$$f(x, W) = h^{(2)}\left(\sum_{i=1}^{D} w_i^{(2)} h^{(1)}\left(\sum_{j=1}^{n} w_{ji}^{(1)} x_j + b_i^{(1)}\right) + b^{(2)}\right)$$

Здесь: x – исходный объект (описанный 11 признаками), x_j – соответствующий признак, n – количество нейронов во входном слое сети, совпадающее с количеством признаков, D – количество нейронов в скрытом слое сети, $w_i^{(2)}$, $w_{ji}^{(1)}$, $b_i^{(1)}$, $b^{(2)}$ – параметры сети, соответствующие весам нейронов, $h^{(1)}$, $h^{(2)}$ – функции активации.

Рис. 27. Отображение формул, подготовленных в LaTeX

На рис. 28, сверху приведен пример использования окружения `\begin{equation}...end {equation}` при наборе формул в ячейке типа `Markdown`, а на рис. 28, снизу – отображение набранных формул после компиляции ячейки.

В табл. 4 представлены часто используемые математические символы и правила их оформления в LaTeX.

```
**Пример 1.**
\begin{equation}
  y = x_0 + \cfrac{1}{x_1
          + \cfrac{1}{x_2
          + \cfrac{1}{x_3 + \cfrac{1}{x_4} } } }
\end{equation}

**Пример 2.**
\begin{equation}
    A_{n,m} =
          \begin{pmatrix}
               a_{1,1} & a_{1,2} & \cdots & a_{1,m} \\
               a_{2,1} & a_{2,2} & \cdots & a_{2,m} \\
               \vdots  & \vdots  & \ddots & \vdots  \\
          a_{n,1} & a_{n,2} & \cdots & a_{n,m}
          \end{pmatrix}
\end{equation}
```

Пример 1.

$$y = x_0 + \cfrac{1}{x_1 + \cfrac{1}{x_2 + \cfrac{1}{x_3 + \cfrac{1}{x_4}}}}$$

Пример 2.

$$A_{n,m} = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,m} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,m} \end{pmatrix}$$

Рис. 28. Запись и отображение формул в LaTeX

Таблица 4. Список математических символов

Строчные и прописные греческие буквы			
α \alpha	ε \epsilon	η \eta	π \pi
β \beta	λ \lambda	μ \mu	σ \sigma
Γ \Gamma	Θ \Theta	Φ \Phi	
Δ \Delta	Σ \Sigma	Ω \Omega	
Символы бинарных отношений			
$\leq$ \leq	$\in$ \in	$\subset$ \subset	$\approx$ \approx
$\geq$ \geq	$\notin$ \notin	$\subseteq$ \subseteq	$\neq$ \neq
Символы бинарных операций			
$\pm$ \pm	$\div$ \div	$\cup$ \cup	$\vee$ \vee
$\cdot$ \cdot	$\times$ \times	$\cap$ \cap	$\wedge$ \wedge
Символы стрелок			
$\rightarrow$ \rightarrow	$\uparrow$ \uparrow	$\Uparrow$ \Uparrow	
$\leftrightarrow$ \leftrightarrow	$\downarrow$ \downarrow	$\Downarrow$ \Downarrow	
$\Leftrightarrow$ \Leftrightarrow	$\updownarrow$ \updownarrow	$\Updownarrow$ \Updownarrow	
Прочие символы			
∞ \infty	$\dots$ \dots	$\forall$ \forall	$\angle$ \angle
$\Re$ \Re	$\cdots$ \cdots	$\exists$ \exists	$\parallel$ \parallel
$\Im$ \Im	$\vdots$ \vdots	$\neg$ \neg	$\perp$ \perp
Математические алфавиты			
$\mathcal{ABC}$ \mathcal{ABC}		$\mathfrak{ABC}$ \mathfrak{ABC}	
$\mathscr{ABC}$ \mathscr{ABC}		$\mathbb{ABC}$ \mathbb{ABC}	

Рассмотрим примеры записи формул в LaTeX и результат их отображения в браузере.

Формула	Результат
`$$\frac{dy}{dx}=\frac{3x^2}{y^3}$$`	$\frac{dy}{dx}=\frac{3x^2}{y^3}$
`$$\arcsin(x^2_{23}\cdot y_2)$$`	$\arcsin(x^2_{23}\cdot y_2)$
`$$\int\limits_0^1{(x^2+4)\cdot dx}$$`	$\int\limits_0^1{(x^2+4)\cdot dx}$
`$$\sqrt[3]{x^2+y^2}$$`	$\sqrt[3]{x^2+y^2}$
`$$x \le \pi$$`	$x \le \pi$
`$$\log_3(x-2)=1$$`	$\log_3(x-2)=1$
`$$\lim \limits_{x \to \infty}(1 + \frac{1}{x^2}) ^ {x^2}= e$$`	$\lim \limits_{x \to \infty}(1 + \frac{1}{x^2}) ^ {x^2}= e$
`$$\sum_{n=1}^{\infty}\frac{(-1)^n}{1+n}=\ln2$$`	$\sum_{n=1}^{\infty}\frac{(-1)^n}{1+n}=\ln2$

Информацию по наборе формул в LaTeX можно посмотреть по ссылке: https://en.wikibooks.org/wiki/LaTeX. Веб сервис *Detexify* (http://detexify.kirelabs.org/classify.html) по нарисованному в браузере символу, который требуется вставить в LaTeX, выдает соответствующую команду для LaTeX.

Примечание. Редактор формул Microsoft Office 2007 и более новых версий также использует команды LaTeX для набора формул. Для многих символов можно найти подсказку, наведя курсор мыши на палитру символов (рис. 29).

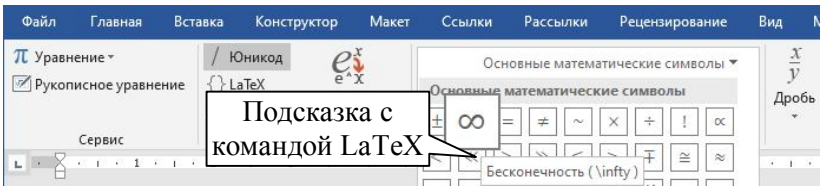


Рис. 29. Отображение подсказок с командами LaTeX в MS Word

Дополнительные возможности кодовых ячеек

Написание кода программы на языке Python в *Jupyter Notebook* производится в ячейки типа `Code`. Рассмотрим основные приемы работы с ячейками типа `Code`, повышающие продуктивность при использовании *Jupyter Notebook*.

Установка сторонних пакетов в Jupyter Notebook

Обычно *Jupyter*-ноутбуки начинаются с ячейки кода, содержащей необходимые импорты с помощью инструкции `import`, после которой указывается название подключаемого модуля (модулей). Пример импорта модулей:

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
```

Перед импортом внешнего модуля необходимо предварительно выполнить процедуру установки (инсталляции) соответствующего пакета с помощью имеющейся (начиная с версии 3.4) в комплекте поставки Python утилиты `pip`, которая самостоятельно найдет запрошенную библиотеку из внешнего репозитория: Github, Bitbucket, Google Code или официального PyPI (*Python Package Index*, реестра пакетов Python), загрузит дистрибутив с этой библиотекой, совместимый с версией Python, и установит ее. Если инсталлируемая библиотека требует для работы другие библиотеки, они также будут установлены.

Jupyter Notebook позволяет запустить установку внешнего пакета прямо из ячейки `Code`, выполнив в ячейке команду вида:

```
!pip install <название пакета>
```

где `<название пакета>` – имя библиотеки, которую требуется установить.

Комментирование кода

Комментарии – это текстовые пояснения, которые находятся непосредственно в самой программе, улучшают понимание кода и не влияют на ход выполнения программы. Во время обработки исходного кода комментарии игнорируются компилятором и интерпретатором. Комментарии позволяют их автору

быстро разбираться в ранее написанном коде, а другим разработчика – значительно улучшить понимание текста программы во время работы над общим проектом.

В Python комментарий пишется с использованием специального символа `#` (решетка), после которого идет текст комментария. Комментарий, содержащийся в строке с выражением, называется *inline* комментарий.

Чтобы закомментировать сразу несколько строк программного кода (или раскомментировать их), надо: 1) выделить нужные строки; 2) нажать комбинацию <Ctrl> + Num </>. Благодаря специальному синтаксису, комментарии выделяются на фоне инструкций, предназначенных для выполнения компьютером.

Вывод значений

Как правило, в ячейке отображается только последний вывод (рис. 30, слева). Для всего остального нужно прописывать `print()`. Это не всегда удобно.

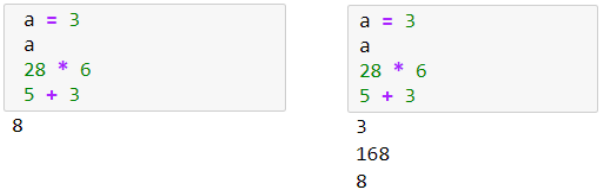


Рис. 30. Вывод результатов в *Jupyter Notebook*
(слева – стандартный режим, справа – настроенный)

Изменить ситуацию можно, запустив в блокноте строки:

```
from IPython.core.interactiveshell import Inter-
            activeShell

InteractiveShell.ast_node_interactivity = 'all'
```

После выполнения этих команд *Jupyter Notebook* будет выводить результат выполнения каждой операции в отдельной строке (рис. 30, справа).

Для возврата к обычному представлению надо выполнить команду:

```
InteractiveShell.ast_node_interactivity =
            'last_expr'
```

Замечание. Запускать команду для изменения настроек необходимо в отдельной ячейке. Тогда изменения вступят в силу при следующем выполнении ячейки.

Магические команды

Одна из крайне удобных составляющих *Jupyter Notebook* – это специальные ***магические команды***, облегчающие процесс разработки и расширяющие возможности. Они используют метаязык *Jupyter Magics*, команды которого обычно начинаются с `%` или `%%`. Строки и ячейки, начинающиеся с `%`, не интерпретируются как обычный код Python.

Магические команды могут быть двух видов:

- *line magics* – начинаются с `%`, выполняются для одной строки;
- *cell magics* – начинаются с `%%`, выполняются для всей ячейки.

Рассмотрим часто используемые магические команды и примеры их вызова.

Контроль времени выполнения

Магические команды `%%time` и `%timeit` используются для измерения времени работы кода.

`%%time` позволяет получить информацию о времени работы кода в рамках одной ячейки (рис. 31).

```
%%time
x = [(i**10) for i in range(100_000)]
```

```
Wall time: 32.9 ms
```

Рис. 31. Пример использования команды `%%time`

`%timeit` запускает переданный ей код по умолчанию 100000 раз и выводит информацию о среднем значении и стандартном отклонении на 7 наиболее быстрых прогонах (рис. 32).

```
%timeit x = [(i**10) for i in range(10)]
```

```
2.31 µs ± 23.2 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

Рис. 32. Пример использования команды `%%timeit`

Команда `%timeit` может принимать аргументы:

- `-n<N>` – количество циклов выполнений;
- `-r<R>` – количество повторений в каждом цикле (каждый из которых состоит из <N> повторений) с выбором наилучшего результата (по умолчанию 7);
- `-p<P>` – точность вывода цифр после.

Например, код ниже выполнится в цикле 2 раза, в каждом из которых он запустится по 3 раза с выбором наилучшего результата. После произойдет расчет среднего и стандартного отклонения:

```
%timeit -n 2 -r 3 x = [(i**10) for i in range(10)]
```

Импорт и экспорт кода

Магическая команда `%%writefile <имя_файла.py>` сохраняет содержимое (код) своей ячейки в файл `<имя_файла.py>`. Если указать только имя файла (а не абсолютный путь), то файл будет создан в той директории, где находится файл ноутбука.

Для загрузки кода из файла напрямую в ячейку используется команда `%load <имя_файла.py>`.

Использование языков внутри блокнота

Jupyter Notebook позволяет модифицировать ячейку в язык с помощью соответствующей магической команды: `%%python2`, `%%ruby`, `%%perl`, `%%html`, `%%latex` и т.д.

Ранее рассматривался вопрос вставки в ячейку типа `Markdown` видео файла. Использование магической команды `%%html` в ячейке типа `Code` позволит добавить в ноутбук файл с видео, воспользовавшись *html*-разметкой, как в примере:

```
%%html
<video width="320" heigth="240" controls>
    <source src="Media/Life.mp4" type="video/mp4">
</video>
```

%lsmagic

`%lsmagic` отображает встроенные магические команды.

Информацию по магическим командам можно найти по адресу https://ipython.org/ipython-doc/3/interactive/magics.html.

Интерактивное управление с помощью виджетов

Для анализа результата выполнения кода в ячейке часто приходится запускать ее несколько раз, немного меня значения входных данных или параметров функций. Это не только непродуктивно, но часто отвлекает от основной задачи анализа данных. Для решения этой проблемы можно использовать инструмент интерактивного управления – `IPywidgets`. Эта библиотека позволяет с помощью виджетов менять переменные, не внося изменений в код, превращая ноутбук из статичного текстового документа в диалоговую панель, удобную для визуализации и работы с данными.

Виджеты – графические элементы, такие как кнопка, выпадающий список или текстовое поле, которые постоянно находится в браузере и позволяет нам контролировать код и данные, реагируя на события и вызов указанных обработчиков. В *Jupyter Notebook* это специальные биты кода, которые связывают воедино код Python, работающий в ядре, и JavaScript/HTML/CSS, работающий в браузере. Их прямое назначение – сделать ноутбук более динамичным, добавить в него различные элементы управления. Наиболее популярные виджеты представлены на рис. 33. Эти графические элементы могут быть собраны и настроены для создания сложных панелей мониторинга.

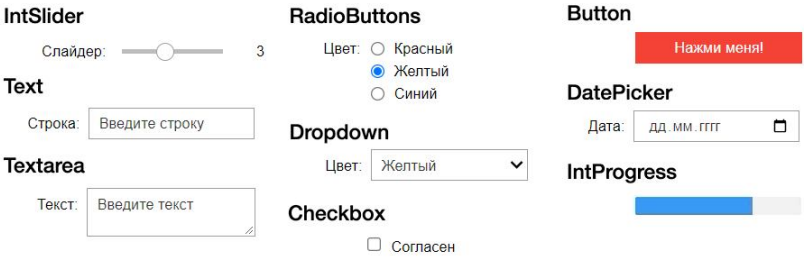


Рис. 33. Часто используемые виджеты

Виджеты могут выполнять код при определенных действиях, позволяя обновлять результат выполнения ячейки без повторного запуска при изменении исходных данных. На рис. 34 показан пример использования виджета `IntSlider`, который создает ползунок и отображает изменяющиеся целые значения.

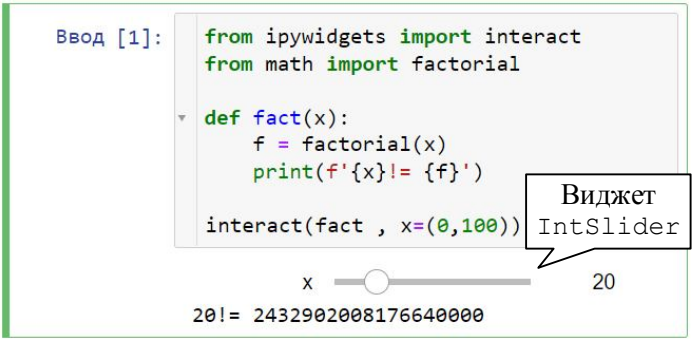


Рис. 34. Пример использования виджета `IntSlider`

`Interact` – самая часто используемая функция библиотеки `ipywidgets`, преобразующая функцию в интерактивный виджет, автоматически создающая элементы управления пользовательского интерфейса. Первый аргумент этой функции – обработчик события, а второй – значение, переданное самому обработчику события (формат: начало, конец, шаг). Она создает слайдер и вызывает некоторую функцию при его изменении. Ему можно придумать массу полезных применений: например, можно смотреть, как ваше решение зависит от изменения какого-то параметра или признака, просто двигая слайдер. Все это реализуется в одну строчку.

Если в `interact` передается кортеж из двух целых чисел (`min, max`), то создается слайдер с целочисленным значением от указанного минимума до максимума (включительно). В этом случае используется размер шага по умолчанию, равный 1. На самом деле, мы можем получить тот же результат, если передадим `IntSlider` в качестве аргумента:

```
interact(f, x=widgets.IntSlider(min=0,
             max=100, step=1, value=20))
```

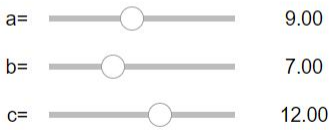
Помимо виджета `IntSlider`, существуют также `FloatSlider` (изменение вещественного числа в диапазоне) и `IntRangeSlider` (изменение целого числа в диапазоне). На рис. 35 показан пример использования виджетов `FloatSlider` при вычислении площади треугольника по трем сторонам по формуле Герона.

```
from ipywidgets import interactive_output
from ipywidgets.widgets import FloatSlider, VBox
from IPython.display import display

def area(a, b, c):
    p = (a + b + c) / 2
    if p < a or p < b or p < c:
        print('Треугольник не существует')
    else:
        s = (p * (p - a) * (p - b) * (p - c)) ** 0.5
        print(f'Площадь треугольника равна {s:0.2f} кв.ед.')

a = FloatSlider(min=0.5, max=20, step=0.1, value=5, description='a=')
b = FloatSlider(min=0.5, max=20, step=0.1, value=5, description='b=')
c = FloatSlider(min=0.5, max=20, step=0.1, value=5, description='c=')
dashboard = VBox([a, b, c])
out = interactive_output(area, {'a': a, 'b': b, 'c': c})

display(dashboard, out)
```




```
Площадь треугольника равна 31.30 кв.ед.
```

Рис. 35. Пример использования виджетов `FloatSlider`

Пояснение. `Interactive_output` обеспечивает дополнительную гибкость: вы можете контролировать расположение элементов пользовательского интерфейса. `VBox` используется для создания контейнера (панели приборов), в котором входные виджеты размещаются друг под другом. Функция `display` отображает объект виджета в ячейке ввода ноутбука.

Для получения полного списка виджетов, которыми можно украсить свой ноутбук, можно выполнить следующую команду:

```
from ipywidgets import widgets
print(dir(widgets))
```

Замечание. Примеры использования различных виджетов можно посмотреть на ресурсе: https://github.com/jupyter-widgets/ipywidgets/blob/master/docs/source/examples/Index.ipynb. Дополнительную документацию по виджетам можно найти по адресу: https://ipywidgets.readthedocs.io/en/latest/.

Дополнительные настройки Jupyter Notebook

Расширения

Несмотря на то, что в *Jupyter Notebooks* встроено множество функций, повышающих продуктивность его использования, можно добавлять новые, с помощью *расширений*. Чтобы ими было удобно управлять создали специальный плагин *NBextensions configurator*, задающий интерфейс для расширений.

Для установки плагина *NBextensions configurator* необходимо выполнить следующие шаги.

Шаг 1. Завершим работу *Jupyter Notebooks*.

Шаг 2. В командной строке ОС или командной строке *Anaconda Prompt* выполним последовательно четыре команды:

```
pip install jupyter_nbextensions_configurator
jupyter_contrib_nbextensions
jupyter contrib nbextension install --user
jupyter nbextensions_configurator enable –user
```

Шаг 3. Запустим *Jupyter Notebooks*.

Шаг 4. После этого в *Jupyter* появилась вкладка *Nbextensions* (рис. 36). Надо переключиться на нее и убрать галочку в первой строке: «*disable configuration for nbextensions without explicit compatibility* (*they may break your notebook environment, but can be useful to show for nbextension development*)».



Рис. 36. Интерфейс *NBextensions* для управления расширениями

Теперь можно одним кликом включать или отключать нужное расширение, при этом ноутбук придется перезагружать.

В самом *Jupyter Notebooks* детально описано, как добавлять, включать и работать с расширениями. Эта информация вместе с GIF-файлом, демонстрирующем возможности расширения, будет доступна под списком после выбора нужного расширения.

В табл. 5 представлены наиболее популярные и востребованные при подготовке ноутбуков расширения.

Таблица 5. Описание популярных расширений

Расширение	**Описание**
`Hinterland`	Добавляет меню автозаполнения кода в ячейке для каждого нажатия клавиши, а не только для клавиши <Tab>
`Autopep8`	Автоматически форматирует Python-код в соответствии с руководством PEP 8 по оформлению кода
`Table of Contents`	Собирает все выполняемые заголовки и отображает их в плавающем окне, в формате боковой панели или в меню навигации
`A Code Prettifier`	Подчищает, форматирует и проставляет отступы
`Code Folding`	В режиме редактирования добавляет иконку-треугольник для скрытия кода. Удобно при наличии длинных функций, которые для читабельности лучше скрывать
`Ruler`	Добавляет визуальную границу ширины кода, показывая красным границу в 80 символов в строке. Плагин нужно поставить обязательно, чтобы при конвертации ноутбука в *pdf* не столкнуться с проблемой, что что-то не влезло в одну строку
`ExecuteTime`	Автоматически запоминает время запуска ячейки сколько времени ушло на выполнение
`Live Markdown Preview`	Включает режим предпросмотра для ячейки типа `Markdown`, при котором еще при вводе содержимого ячейки под ней будет отображаться результата ввода
`Variable Inspector`	Отображает информацию об используемых переменных: имя, тип, размер, значение

Замечание. Для работы расширения `Autopep8` необходимо, чтобы на локальной машине (из командной строки *Anaconda Prompt*) была выполнена команда:

```
pip install autopep8 --user
```

Для преобразования написанного кода в соответствии со стилевым руководством PEP8 на панели инструментов появиться иконка в виде молота  .

Поскольку задачи, решаемые в *Jupyter Notebook*, разнообразны, то нет общего списка расширений, какие необходимо ставить. Его лучше всего определять на собственном опыте, индивидуально.

На рис. 37 показан внешний вид окна *Jupyter Notebooks* с установленными расширениями (`Table of Contents`, `Autopep8`, `Variable Inspector`, `ExecuteTime`).

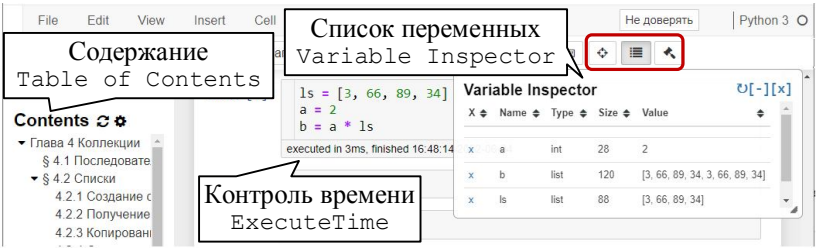


Рис. 37. *Jupyter Notebooks* с дополнительными расширениями

Изменение темы оформления

Помимо функциональности при работе *Jupyter Notebook* многих интересует и эстетическая сторона этого инструмента, а именно тема оформления, используемая в редакторе. Для многих программистов важно при работе использовать темную тему, которая «бережет» глаза.

Для изменения темы оформления *Jupyter Notebook* выполним следующий шаги.

Шаг 1. Установим пользовательские темы для ноутбука (из командной строки *Anaconda Prompt*), выполнив команду:

```
pip install jupyterthemes
```

Шаг 2. После шага 1 можно промотреть имена установленных пользовательских тем, выполнив команду:

```
jt -l
```

Названия тем отобразятся в столбец, как на рис. 38:

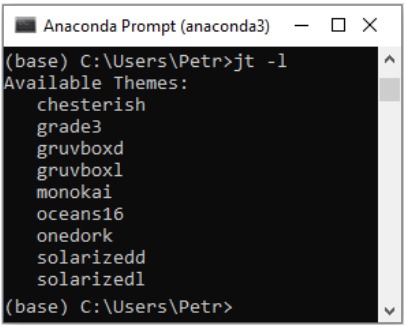


Рис. 38. Список тем оформления для *Jupyter Notebook*

Шаг 2. Теперь можно выбирать название темы и выполнить команду `jt -t <название темы>`. Например, для изменения темы на темную:

```
jt -t onedork
```

Шаг 3. Для изменения темы в уже запущенном ноутбуке достаточно обновить браузер, воспользовавшись кнопкой ↻.

Примечание. Если требуется восстановить исходную тему, установленную по умолчанию, введите следующий код:

```
jt -r
```

Управление папками и ноутбуками в Jupyter

На домашней странице (*Home Page*) сервера *Jupyter* после установки будут доступны три вкладки: *Files*, *Running* и *Clusters* (вкладка *NBextensions* появилась после установки соответствующего плагина).

Вкладка *Files* (рис. 39) предоставляет доступ к файлам и папкам в текущем каталоге, из которого было вызвано приложение *Jupyter Notebook*. Все папки в каталоге помечены иконкой с папкой, ноутбуки – иконкой записной книжки. У запущенных в работу ноутбуков будет зеленая иконка (и при этом

рядом с ними написано *Running*/ *Запустить*), у нерабочих – серая. Файлы и папки открываются стандартно – двойными кликом по ним в браузере. Эта вкладка содержит кнопки, позволяющие создавать, удалять, переименовывать, скачивать, копировать файлы и директории.

С помощью кнопки *Upload* (*Загрузить*) можно загрузить файл на сервер ноутбука (рис. 39). Ее удобно использовать при загрузке, например, файлов с данными для анализа.



Рис. 39. Вкладка *Files*

Ранее уже рассматривались вопросы создания файлов и папок в *Jupyter Notebook* с помощью кнопки *New* и последующим выбором соответствующей команды (рис. 39). Здесь находится также команда *Terminal* (*Терминал*), которая запускает терминал вашей операционной системы в браузере, где можно выполнять необходимые для работы команды.

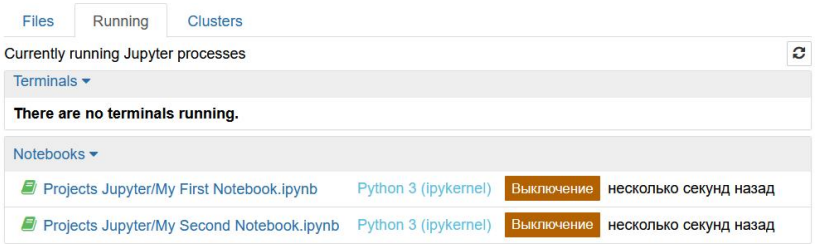


Рис. 40. Вкладка *Running*

Вкладке *Running* (рис. 40) отображает работающие в текущий момент ноутбуки и используемые ими ядра. Эта вкладка окажется довольно полезной, когда требуется выключить сервер, но нужно убедиться, что все данные сохранены.

Контрольные вопросы

1. Что такое *Jupyter Notebook*?
2. Как называются файлы, созданные в *Jupyter Notebook*, и какое они имеют расширение?
3. Что такое Markdown?
4. Что такое LaTeX?
5. Что понимают под ядром (Kernel) в *Jupyter Notebook*?
6. Какие типы ячеек можно использовать в *Jupyter Notebook*?
7. Каким образом можно запустить *Jupyter Notebook*?
8. Из каких элементов состоит окно браузера после запуска *Jupyter*?
9. Как в *Jupyter Notebook* создать папку?
10. Какие типы файлов можно создать в *Jupyter Notebook*?
11. Как в *Jupyter Notebook* создать новый ноутбук?
12. Каким образом в *Jupyter Notebook* переименовать папку/файл?
13. Какие режимы работы с ячейками существуют и как узнать текущий режим работы ячейки?
14. Каким образом можно запустить ячейку на выполнение?
15. Что означает нелинейная последовательность выполнения команд? Приведите пример.
16. Как добавить новую ячейку с помощью панели инструментов и комбинацией клавиш?
17. Как поменять порядок следования ячеек в ноутбуке?
18. Что такое контрольная точка?
19. Перечислите основные «горячие» клавиши, используемые при работе в *Jupyter Notebook* и их назначение.
20. Как посмотреть список «горячих» клавиш *Jupyter Notebook*?
21. Как получить подсказку по командам и используемым объектам в программе?
22. Перечислите кнопки панели инструментов *Jupyter Notebook* и их назначение.
23. Как пронумеровать строки в ячейке?
24. Как изменить тип ячейки?
25. Охарактеризуйте команды меню Kernel.

26. Приведите примеры разметки Markdown, используемой для форматирования текста и заголовков.
27. Как в разметке Markdown создать таблицу?
28. Как добавить в ноутбук ссылку на Интернет-ресурс?
29. Как добавить изображение в ноутбук?
30. Как добавить формулу в ноутбук?
31. Какие режимы вставки формул вам известны? Приведите примеры их использования.
32. Как установить в *Jupyter Notebook* внешнюю библиотеку
33. Что такое магическая команда и какие виды магических команд вы знаете?
34. Приведите примеры использования магических команд.
35. Что такое виджеты и какие виджеты часто используются?
36. Приведите примеры использования виджетов в программе.
37. Что понимают под расширением для *Jupyter Notebook*?
38. Какие расширения позволяют повысить продуктивность использования *Jupyter Notebook*?
39. Как изменить тему оформления в *Jupyter Notebook*?
40. Как можно импортировать файлы в *Jupyter Notebook*?

Задания

Во всех вариантах заданий подготовьте ноутбук с отчетом к лабораторной работе, в который включите:

1) текст задания своего варианта (выражение для нахождения значения функции подготовьте в формате LaTeX);
2) рисунок с блок-схемой решения задачи, подготовленный с использованием графического редактора, содержащего инструменты для создания блок-схем (например, *Microsoft Visio* или бесплатный онлайн-редактор *Draw.io*, доступный по адресу https://www.draw.io/);
3) программу решения задачи, подготовленную в двух вариантах: в первом для ввода данных используйте функцию `input()`, во втором для задания значений входных переменных используйте виджеты;
4) расчет для трех тестовых наборов входных данных, выполненный с использованием компьютерных программ

для математических и технических расчетов, например, *MathCAD*, *WolframAlpha* или *SMath Studio* (скрин результатов расчета из программы);

5) расчет контрольных значений для пяти наборов входных данных, включающий наборы из п.4, представленный в виде таблицы, округлив результат вычисления значения функции до четырех знаков после запятой.

При подготовке ноутбука используйте форматирование текста и заголовки.

Переконвертируйте созданный ноутбук в *pdf*-файл.

На рис. 41 представлена блок-схема для нахождения значения функции y, заданной выражением:

$$y = \begin{cases} \sum_{k=1}^{10} x^k \cos\frac{k\pi}{3}, & \text{если } x \ge a; \\ \prod_{m=1}^{6} (a^m + x^m), & \text{если } x < a. \end{cases}$$

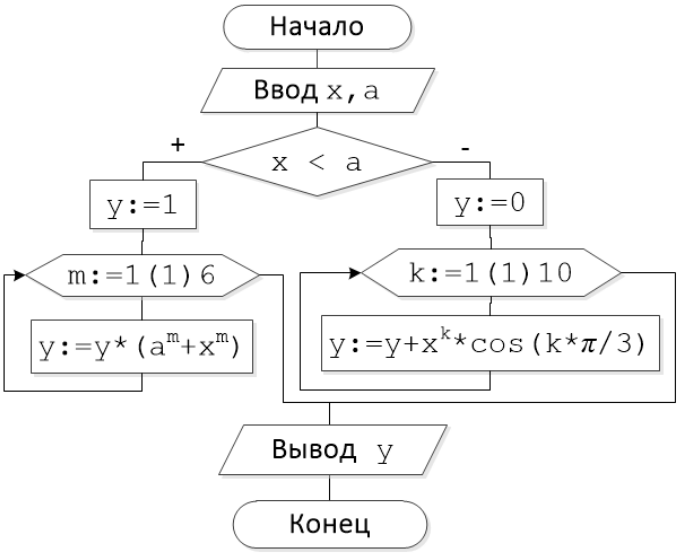


Рис. 41. Блок-схема решения задачи

На рис. 42 представлены результаты расчета значений, выполненные в математическом пакете *MathCAD*, а на рис. 43 –

результаты работы программы из *Jupyter Notebook*, представленные в виде таблицы.

$$y(x,a) := \text{if}\left[x < a,\ \prod_{m=1}^{6}\left(a^m + x^m\right),\ \sum_{k=1}^{10}\left(x^k \cdot \cos\left(\frac{k\cdot\pi}{3}\right)\right)\right]$$

$y(-2,1.5) = -560131.6026$ $\qquad$ $y(-1,-1) = -0.5$

$y(2.6,-1.3) = -12804.5854$ $\qquad$ $y(0,1.2) = 46.0051$

Рис. 42. Расчет тестовых значений в *MathCAD*

x	a	y(x, a)
-2	1.5	-560131.6026
2.6	-1.3	-12804.5854
-1	-1	-0.5
0	1.2	46.0051
1.2	-2.3	-7.2686

Рис. 43. Расчет контрольных значений

Вариант 1. Вычислить

$$y = \begin{cases} \sum_{k=1}^{10} x^k \sin\dfrac{k\pi}{4}, & \text{если } x \ge a; \\ \prod_{m=1}^{5} (a^m - x^m), & \text{если } x < a. \end{cases}$$

Вариант 2. Вычислить произведение

$$P = \prod_{i=1}^{10} \frac{i(i+a)}{i^2 + a^2} + \prod_{k=2}^{4} \frac{k^3}{k+a}.$$

Вариант 3. Вычислить сумму

$$S = \sum_{n=0}^{100} (-1)^n \frac{x^{2n}}{(2n)!}.$$

Вариант 4. Вычислить

$$y = \frac{\sin x + 2}{3 + \cos x} \sum_{n=0}^{20} a x^n; \quad a = \begin{cases} 2n, & \text{если} \quad x \le 0,5; \\ \dfrac{n}{2}, & \text{если} \quad x > 0,5. \end{cases}$$

Вариант 5. Вычислить

$$z = \sum_{k=0}^{10} \ln x \cdot \sin k(x - a); \quad a = \begin{cases} \pi/4, & \text{если} \quad x \le 1; \\ \pi, & \text{если} \quad x > 1. \end{cases}$$

Вариант 6. Вычислить

$$y = \begin{cases} \displaystyle\sum_{k=1}^{10} x^k a^k, & \text{если } x \ge a; \\ \displaystyle\prod_{m=1}^{8} (a^m - x^m), & \text{если } x < a. \end{cases}$$

Вариант 7. Вычислить сумму

$$\alpha = \sum_{k=1}^{10} \left(k + a \ln x\right); \quad a = \begin{cases} -1, & \text{если} \quad x \le 1; \\ 1, & \text{если} \quad x > 1. \end{cases}$$

Вариант 8. Вычислить

$$z = \begin{cases} \displaystyle\prod_{n=1}^{8} \left(\frac{x}{n}\right)^n, & \text{если} \quad x \le 2; \\ \displaystyle\sum_{n=0}^{5} (1 + xn), & \text{если} \quad x > 2. \end{cases}$$

Вариант 9. Вычислить произведение

$$y = \prod_{n=1}^{8} \left(p - \frac{x^n}{2n+1}\right); \quad p = \begin{cases} 1, & \text{если} \quad n \le 5; \\ 2, & \text{если} \quad n > 5. \end{cases}$$

Вариант 10. Вычислить

$$y = \frac{a x^2}{\sqrt{x + a}}; \quad a = \begin{cases} \displaystyle\sum_{k=1}^{12} \frac{2kx}{x + k^2}, & \text{если} \quad x < 1; \\ 1, & \text{если} \quad x \ge 1. \end{cases}$$

Вариант 11. Вычислить сумму

$$S = \sum_{n=0}^{20} \frac{2\left(x-1\right)^{2n-1}}{\left(2n+1\right)\left(x+1\right)^{2n+1}}.$$

Вариант 12. Вычислить произведение

$$\lambda = \prod_{n=1}^{5}\left(\frac{1}{n} + a\sin x\right); \quad a = \begin{cases} 1, & \text{если} \quad x \ge 0; \\ -1, & \text{если} \quad x < 0. \end{cases}$$

Вариант 13. Вычислить

$$z = \begin{cases} \dfrac{x+1}{2}\displaystyle\sum_{n=1}^{10}\left(\frac{x}{n}\right)^n, & \text{если} \quad x \le 2; \\ \dfrac{\sin x + \cos x}{2+\sin x}\displaystyle\prod_{n=0}^{15}\left(1+\frac{x}{n+2}\right), & \text{если} \quad x > 2. \end{cases}$$

Вариант 14. Вычислить произведение

$$P = \prod_{n=1}^{5}\left(n + \frac{\sin x}{n}\right).$$

Вариант 15. Вычислить сумму

$$S = \sum_{n=0}^{20} \frac{x^{2n+1}}{\left(2n+1\right)!}.$$

Вариант 16. Вычислить

$$z = \begin{cases} \displaystyle\prod_{n=1}^{8}\left(\frac{x}{n}\right)^n, & \text{если} \quad x \le 1; \\ \displaystyle\sum_{n=1}^{5}(3 + xn), & \text{если} \quad x > 1. \end{cases}$$

Вариант 17. Вычислить

$$y = \frac{x+2}{1+\cos x}\sum_{n=1}^{10} a\left(1+x^n\right); \quad a = \begin{cases} 2n, & \text{если} \quad x \le 1; \\ \dfrac{n}{2}, & \text{если} \quad x > 1. \end{cases}$$

Вариант 18. Вычислить сумму

$$S = \sum_{k=0}^{20} \frac{2 + \sin\left(\frac{\pi}{k} x\right)}{3 + \cos\left(\frac{\pi x}{k+2}\right)}.$$

Вариант 19. Вычислить

$$z = \frac{a+1}{a + 2{,}5 \cdot 10^{-3}} \sum_{n=0}^{15} \frac{a^{2n}}{(n+1)!}.$$

Вариант 20. Вычислить

$$z = \begin{cases} \prod_{n=1}^{5} \left(n + \frac{\sin x}{n}\right), & \text{если} \quad x \le 2; \\ \sum_{n=0}^{5} (1 + xn), & \text{если} \quad x > 2. \end{cases}$$

Вариант 21. Вычислить произведение

$$\lambda = \prod_{n=1}^{10} \left(1 + \frac{ax}{n+2}\right); \quad a = \begin{cases} 1, & \text{если} \quad x \ge 0; \\ -1, & \text{если} \quad x < 0. \end{cases}$$

Вариант 22. Вычислить сумму

$$z = \sum_{n=0}^{10} \frac{(-1)^{2n} x^{2n}}{(2n+1)!}.$$

Вариант 23. Вычислить

$$z = \sum_{k=1}^{15} x \cdot \cos k(x + a); \quad a = \begin{cases} \pi/3, & \text{если} \quad x \le 1; \\ \pi, & \text{если} \quad x > 1. \end{cases}$$

Вариант 24. Вычислить

$$y = \frac{ax^2}{\sqrt{|x + a|}} \sum_{k=1}^{10} \frac{2kx}{x + k}; \quad a = \begin{cases} 1, & \text{если} \quad x < 1; \\ -1, & \text{если} \quad x \ge 1. \end{cases}$$

Вариант 25. Вычислить произведение

$$P = \prod_{n=0}^{10} \left(2n + \frac{\sin x}{n+1}\right).$$